

Notebook

February 8, 2026

1 Атака на подгруппы в протоколе Диффи-Хеллмана

1.1 Короткое введение

Протокол Диффи-Хеллмана дает возможность двум сторонам обмениваться ключевой информацией так, что любой неактивный (без возможности влиять на систему) злоумышленник не может получить эту информацию.

Высокоуровневое представление ДН:

1. Алиса создает закрытый ключ a , открытый ключ A , отправляет A Бобу
2. Боб создает закрытый ключ b , открытый B , отправляет B Алисе
3. Алиса и Боб вырабатывают новый (одинаковый) ключ из (a, B) и (b, A) .

Любой неактивный злоумышленник получает открытые ключи (A, B) , но вычисление $(a, B) = (b, A) \Rightarrow K$ на их основе - задача в NP.

1.2 Пример

Один из способов генерации таких пар (a, A) , (b, B) - использование мультипликативных групп по модулю простого числа (обычно обозначается Z_p^*). Z_p^* состоит из чисел от 1 до $p - 1$, т.е. общее число элементов или мощность группы Z_p^* равна $p - 1$.

Посмотрим на пример: Z_5^*

Таблица умножения:

	1	2	3	4
1	1	2	3	4
2	2	4	1	3
3	3	1	4	2
4	4	3	2	1

Заметно, что:

- 1 - нейтральный элемент (любой элемент, умноженный на 1 остается тем же элементом)

- 2 - обратный элемент к 3, а 4 - сам к себе

Также можно заметить, что это циклическая группа (что нормально для Z_p^*), значит можно получить все элементы группы последовательным умножением одного элемента (генератора) на себя. В данном случае один из возможных генераторов - элемент 2:

- $2 = 2$
- $2 * 2 = 4$
- $4 * 2 = 3$
- $3 * 2 = 1$

Так каким образом работает ДН в Z_p^* ? Об используемых простом числе p и генераторе g договариваются заранее (обычно они часть стандарта).

1. Алиса и Боб выбирают случайные $a, b \in 2, p-2$ (Алиса выбирает a , Боб выбирает b)
2. Алиса вычисляет $A = g^a \bmod p$, Боб вычисляет $B = g^b \bmod p$
3. Они обмениваются A и B
4. Алиса вычисляет $C = B^a \bmod p = g^{ab} \bmod p$
5. Боб вычисляет $C = A^b \bmod p = g^{ba} \bmod p$

1.3 Проблемы

В общем случае злоумышленнику сложно получить такое $a = \log(A)$, что $A = g^a \bmod p$, но есть исключения. Мультипликативные группы по модулю простого числа содержат подгруппы (вообще группы содержат подгруппы). В лучшем случае кроме самой группы существует только три подгруппы: $1, 1, p-1$ и ещё одна подгруппа мощностью $\frac{p-1}{2}$. Такое возможно, когда p - безопасное простое число ($\frac{p-1}{2}$ тоже простое).

Для каждого делителя d ($d|p-1$) порядка (мощности) группы существует подгруппа H группы Z_p^* ($H < Z_p^*$), такая что её порядок равен d ($|H| = d$). Например, в случае $p = 13$, $p-1 = 12$ делится на 1, 2, 3, 4, 6, 12 Поэтому есть подгруппы степеней 2, 3, 4, 6 В данном случае 2 - генератор.

Давайте посмотрим на генерируемую циклическую группу:

```
[ ]: p=13
for i in range(p):
    print("2^{0}={1}".format(i,pow(2,i,p)))
```

Как же найти генератор для подгруппы заданного размера? Это очень легко. Для подгруппы порядка d , генератором будет $g_d = g^{\frac{p-1}{d}} \bmod p$. Логично, что при возведении g_d в степень d результат равен $g_d^d = g^{\frac{p-1}{d} * d} \bmod p = g^{p-1} \bmod p = 1 \bmod p$

Мы можем выразить закрытый ключ a как $a = k * d + r$, где $r < d$, $k \in N$. Если мы возведём открытый ключ $A = g^a \bmod p$ в степень $\frac{p-1}{d}$, результат будет равен

$$A^{\frac{p-1}{d}} \bmod p = g^{a \cdot \frac{p-1}{d}} \bmod p = g^{(k \cdot d + r) \cdot \frac{p-1}{d}} \bmod p = g^{k \cdot d \cdot \frac{p-1}{d} + r \cdot \frac{p-1}{d}} \bmod p = g^{k \cdot (p-1) + r \cdot \frac{p-1}{d}} \bmod p = g^{r \cdot \frac{p-1}{d}} \bmod p.$$

Сравнивая $A^{\frac{p-1}{d}} \bmod p$ со всеми возможными $g_d^l \bmod p$, $l \in \{0, \dots, d-1\}$, мы можем найти r .

Давайте попробуем:

```
[ ]: a=7
g=2
A=pow(g,a,p)
d=4
g_d=pow(g,(p-1)//d,p)
Ap=pow(A,(p-1)//d,p)
print ('A^{0}={1}'.format((p-1)//d,Ap))
for i in range(d):
    cur_pow=pow(g_d,i,p)
    if Ap!=cur_pow:
        print('(g_{0})^{1}={2}!=A^{((p-1)/d)}'.format(d,i,pow(g_d,i,p)))
    else:
        print('(g_{0})^{1}={2}=A^{((p-1)/d)}'.format(d,i,pow(g_d,i,p)))
    print ('a={0} mod {1}'.format(i,d))
```

Таким образом достаточно просто восстановить пары $r_i \equiv a \pmod{d_i}$ для $p = 13$. Каким образом получить a из r_i ? Китайская Теорема об Остатках гласит, что это возможно, если все d_i взаимно простые и их произведение превышает a (т.е. нам надо использовать взаимно простые делители).

```
[ ]: try:
    from gmpy2 import invert
except ImportError:
    try:
        from Crypto.Util.number import inverse as invert
    except ImportError:
        print ("You need to install either gmpy2:\nsudo apt install_\npython-gmpy2\nor pycryptodome:\npython3 -m pip install_\npycryptodome")
        raise Exception
def crt(remainders,modules,M):
    result = 0
    for (a, b) in zip(remainders,modules):
        result = (result+a*((M)//b)*invert((M)//b, b)) % (p-1)
    return result
```

Давайте попробуем для $a = 7$, $p = 13$ Мы знаем, что:

- $a \equiv 3 \pmod{4}$
- $a \equiv 1 \pmod{3}$

```
[ ]: remainders=(1,3)
modules=(3,4)
m=p-1
print ('a={0}'.format(crt(remainders,modules,m)))
```

В общем случае мы можем разложить $p-1 = p_1^{k_1} * \dots * p_n^{k_n}$

Случай, когда все p_i малы, а $k_i = 1$, самый простой. Но что если все p_i малы, но есть несколько $k_i \neq 1$, такие что прямой перебор подгруппы порядка $p_i^{k_i}$ занимает слишком много времени?

Сначала надо решить $a \equiv r \pmod{p_i}$.

Теперь нам известно, что $a = k * p_i + r$, $k \in \{0\} \cup N$, где k - неизвестно. Нам надо найти $a = r_1 \pmod{p_i^2}$

Можно переписать выражение как $a = k_1 * p_i^2 + k_2 * p_i + r$, $k_2 \in \{0, \dots, p_i - 1\}$, $k_1 \in \{0\} \cup N$

$$r_1 = k_2 * p_i + r$$

Давайте посмотрим, что происходит при возведении A в $\frac{p-1}{p_i^2}$

$$A^{\frac{p-1}{p_i^2}} \pmod{p} = g^{a * \frac{p-1}{p_i^2}} \pmod{p} = g^{(k_1 * p_i^2 + k_2 * p_i + r) * \frac{p-1}{p_i^2}} \pmod{p} = g^{k_1 * p_i^2 * \frac{p-1}{p_i^2} + k_2 * p_i * \frac{p-1}{p_i^2} + r * \frac{p-1}{p_i^2}} \pmod{p} = g^{k_1 * (p-1) + k_2 * \frac{p-1}{p_i} + r * \frac{p-1}{p_i^2}} \pmod{p} = g^{k_2 * \frac{p-1}{p_i} + r * \frac{p-1}{p_i^2}} \pmod{p}.$$

$$\text{Значит } A^{\frac{p-1}{p_i^2}} \pmod{p} = g^{k_2 * \frac{p-1}{p_i} + r * \frac{p-1}{p_i^2}} \pmod{p} = g^{k_2 * \frac{p-1}{p_i}} * g^{r * \frac{p-1}{p_i^2}} \pmod{p}.$$

Единственная неизвестная - это k_2 , которая определена в небольшом интервале и её можно перебрать.

Рассмотрим пример для $p = 19$:

```
[ ]: import random
p=19
a= random.randint(2,p-2)
g=2
A=pow(g,a,p)
print ("Initial a={0},A={1}".format(a,A))
d=3
A_d=pow(A,(p-1)//d,p)
g_d=pow(g,(p-1)//d,p)
print('g^{0}={1}'.format((p-1)//d,g_d))
x=1
r_d=-1
for i in range(d):
    if x==A_d:
        r_d=i
        break
    x=(x*g_d)%p
print ('a={0} mod {1}'.format(r_d,d))
```

```

d2=d*d
A_d2=pow(A, (p-1)//d2,p)
g_d2=pow(g, (p-1)//d2,p)
g_needed=pow(g_d2,d,p)

x=pow(g_d2,r_d,p)
for i in range(d):
    if x==A_d2:
        k_d2=i
        break
    x=(x*g_needed)%p
r_d2=k_d2*d+r_d
print ('a={0} mod {1}'.format(r_d2,d2))

```

Теперь мы знаем такие r , что $a \equiv r \pmod{9}$. Всё, что осталось сделать, это найти r для $a \equiv r \pmod{2}$

```

[ ]: remainders=[r_d2]
modules=[d2]
d=2
A_d=pow(A, (p-1)//d,p)
g_d=pow(A, (p-1)//d,p)
x=1
for i in range(d):
    if x==A_d:
        r=i
        break
    x=(x*g_d)%p
remainders.append(r)
modules.append(d)
print ('Computed a={0}'.format(crt(remainders,modules,p-1)))

```

1.4 Задание

Теперь вы готовы решить задание. Можете воспользоваться API для общения с сервером или подсоединиться к нему при помощи netcat. Сервер выдаст вам модуль и открытый ключ A . Используемый генератор - 2. Ваша цель - найти закрытый ключ a . Сначала нужно разложить $p - 1$ на множители (все простые множители меньше 65536), потом отобразить A в подгруппы, как мы это делали ранее. Удачи!

```

[ ]: import socket
import re
class VulnServerClient:
    def __init__(self, show=True):
        """Инициализация, подключаемся к серверу"""

```

```

self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
self.s.connect(('cryptotraining.zone',1345))

def recv_until(self,symb=b'\n>'):
    """Получаем сообщения от сервера, по умолчанию до_
    ↪приглашения"""
    data=b''
    while True:
        data+=self.s.recv(1)
        if data[-len(symb):]==symb:
            break
    return data
def getChallenge(self,show=True):
    data=self.recv_until()
    try:
        data=data.decode()
    except UnicodeDecodeError:
        print ('Ошибка декодирования юникода. Попробуйте_
    ↪переподключиться к серверу.')
        return (None,None)
    if show:
        print (data)
    p=int(re.search(r'(?<=p=)\d+',data).group(0))
    A=int(re.search(r'(?<=2\**k \(\mod p\)=)\d+',data).group(0))
    return (p,A)

def checkSolution(self,k, show=True):
    self.s.sendall((str(k)+'\n').encode())
    data=self.recv_until(b'\n')
    try:
        data=data.decode()
    except UnicodeDecodeError:
        print ('Ошибка декодирования юникода. Попробуйте_
    ↪переподключиться к серверу.')
        return None
    if show:
        print (data)
    if data.find('flag')!=-1:
        return True
    else:
        data=self.recv_until(b'>')
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Ошибка декодирования юникода. Попробуйте_
    ↪переподключиться к серверу.')

```

```
        return None
    if show:
        print (data)
    return False
def __del__(self):
    self.s.close()

vs=VulnServerClient()
(p,A)=vs.getChallenge()
vs.checkSolution(1)
```

[]: